THE EYES — Interim Report

Multi-View Perception and Scene Understanding for Robot Manipulation

NUS-ISS Pattern Recognition Systems Practice Module — Interim Report (DRAFT)

Field	Detail
Project Title	The Eyes — Multi-View Perception and Scene Understanding for Robot Manipulation
Group ID / Cohort	Group 12 / AIS08
Team Members	Chan Wai Jye (A0340644J)
Status as of	2026-09-06
Companion document	`PRS-PM-2026-AIS08-GRP-12-TheEyes-Proposal.md`

0. Purpose and scope of this document

This is the accumulating record of **measured results**. It exists so the proposal can state the plan and this document can carry the evidence, and it grows into the final report's *system performance* and *findings and discussion* chapters (proposal §11.1).

Two rules govern everything below, both from `CLAUDE.md`:

- **Every figure comes from an actual run.** Nothing here is projected. Anything unresolved carries a `[VERIFY]` marker rather than a plausible guess.
- **Figures marked *synthetic* come from generated input** and are not evaluation results. Every tool that prints them says so.

Results are reported whichever way they come out. §4.4 is a negative result on the project's own multimodal claim and is presented as one.

1. Implementation status

Reported from actual runs; nothing in this table is projected. Figures marked *synthetic* come from generated input and are not evaluation results.

Module	State	Evidence
`camera`	Built, **verified on hardware**	Both D435i at 30.0 fps, 848×480 aligned; hardware-synced pair jitter **0.14–1.1 ms sigma** (33 ms budget); selection by serial survived four replugs
`calibration`	Built, **run on the real rig 2026-09-06, recalibrated 2026-09-09**	Current extrinsics: reprojection RMSE **0.47 / 0.41 px**, round trip 0.12 mm mean against a 5 mm gate. Recalibration was forced by a physical move the camera-to-camera invariant caught: baseline 749.9 → 706.7 mm, relative rotation 81.96° → 76.77° (design 737 mm, 80°)
`detector`	Built, **trained and deployed**	`synth50k_v2`, 40 epochs, ONNX, C++/Python parity re-verified. GPU **p50 3.60 / 3.43 ms per view** (12.60 / 12.37 CPU), same detections either provider
`pose`	Built, **verified end-to-end**	Drives the real FoundationPose server: 30/30 frames, p50 11.1 ms end to end
`fusion` ★	Built, **first real-object run 2026-09-06**	Eight-arm ablation harness (below, *synthetic*). 150 live pairs of a printed cube: both views fused on every pair, 0 gate rejections, per-view disagreement 11 mm
`scenegraph` ★	Built	Emits PDDL `(:init …)`; contract-tested against fusion
`runtime`	**Built and run on hardware 2026-09-06**	Whole chain in one C++ process, live with the real pose server: **p50 51.7 ms per pair**, p95 78 ms. Bit-exact record/replay, SQLite metrics per pair
`clustering`, `twin`	Not started	Above MVP; documented cut order
object meshes	**Acquired**	All three classes present, re-centred, config tied to geometry by test (proposal §6.2)

336 automated tests pass across eight suites under `-Wall -Wextra -Wpedantic`; every module has a mock mode needing no hardware.

2. System design as built

2.1 Runtime process model

The delivered runtime is **one C++ process, not ROS2** (decided 2026-09-06 with the live runner). The six stages pass the `INTERFACES.md` structs by function call rather than as messages. For a fixed pipeline on one machine this costs nothing and avoids serialisation on the hot path and a second build system. Nor does anything downstream need ROS2: *The Planner* ships none — its bridge is FastAPI (verified 2026-09-08). `INTERFACES.md` still specifies `scene_state` and the PDDL facts as the boundary, so a ROS2 publisher remains a thin adapter over the same structs.

```
the_eyes_run check   # configs, model, pose server, cameras
the_eyes_run live    # cameras by serial -> RGB-D YOLOv8 -> FoundationPose
                     #   -> association -> fusion -> scenegraph -> PDDL
the_eyes_run replay  # bit-exact replay of a recorded session
```

2.2 Fusion parameters as configured

From `config/fusion.yaml`. These are the constants that shape the estimate, held outside the code because the module is written from first principles:

Parameter	Value	Why
`association.max_distance_m`	0.15	Two same-class per-view estimates merge below this. Upper bound is the closest same-class spacing in the benchmark scenes
`disagreement_gate.max_translation_m`	0.05	A view further than this from consensus is dropped. **Must** stay below the association radius, or it can never fire — enforced by `LoadFusionConfig`
`disagreement_gate.max_rotation_deg`	30.0	As above, rotational
`two_view_conflict_confidence_penalty`	0.5	With two cameras there is no majority; the higher-scoring view is kept and confidence halved, recording "we had a conflict and guessed"
`calibration.max_reprojection_rmse_px`	0.5	Fusion refuses to start above this. Bad extrinsics poison every fused pose while leaving each self-consistent
`ekf.max_coast_s`	1.0	A track unseen this long is dropped and its id retired

2.3 Detector network, layer by layer

Read from `runs/detector/rgbd_50k_v2/weights/best.pt` with a forward pass at 1×4×640×640, so the shapes are observed rather than derived from the YAML. Design rationale is in proposal §5.5; this is the as-built record.

#	Module	Output	Params
0	`Conv` **stem, 4-channel**	1×16×320×320	608
1	`Conv`	1×32×160×160	4,672
2	`C2f`	1×32×160×160	7,360
3	`Conv`	1×64×80×80	18,560
4	`C2f` → *neck skip*	1×64×80×80	49,664
5	`Conv`	1×128×40×40	73,984
6	`C2f` → *neck skip*	1×128×40×40	197,632
7	`Conv`	1×256×20×20	295,424
8	`C2f`	1×256×20×20	460,288
9	`SPPF`	1×256×20×20	164,608
10–11	`Upsample` + `Concat`(6)	1×384×40×40	0
12	`C2f`	1×128×40×40	148,224
13–14	`Upsample` + `Concat`(4)	1×192×80×80	0
15	`C2f` → **P3 head**	1×64×80×80	37,248
16–17	`Conv` s2 + `Concat`(12)	1×192×40×40	36,992
18	`C2f` → **P4 head**	1×128×40×40	123,648
19–20	`Conv` s2 + `Concat`(9)	1×384×20×20	147,712
21	`C2f` → **P5 head**	1×256×20×20	493,056
22	`Detect`(15, 18, 21)	8,400 anchors	751,897
	total		**3,011,577**

Backbone 1.27 M, neck 0.99 M, head 0.75 M. The head is the single largest module at 25% of the network — worth knowing before any pruning: the backbone is not where the parameters are.

The 8,400 anchors are $80^2+40^2+20^2$ at strides 8/16/32. With `reg_max = 16` the raw head emits 67 channels per anchor (4 box edges × 16 DFL bins + 3 classes); ONNX export decodes the distribution to four coordinates, giving the deployed `1×7×8400`.

The depth channel's cost. The 4-channel stem is `Conv(16, 4, 3×3, s2)`, and it is the only layer that differs from the RGB-only ablation control:

	stem weights	total parameters
RGB-only (ablation control)	432 — `(16,3,3,3)`	3,011,433
RGB-D (deployed)	**576 — `(16,4,3,3)`**	**3,011,577**
difference	+144	**+144 (+0.005%)**

2.4 Pose networks

FoundationPose ships two checkpoints, both frozen. Parameter counts and module groups read directly from the `.pth` files.

	Refiner	Scorer
checkpoint	`2023-10-28-18-33-37`	`2024-01-11-20-02-45`
parameters	**17,043,797**	**15,986,512**
per-branch encoder	685,894	685,894
joint encoder	12,994,057	12,994,057
positional embedding	204,800	204,800
task modules	`trans_head` 1,579,523 + `rot_head` 1,579,523	`att` 1,050,624 + `att_cross` 1,050,624
output	`Linear(512→3)` ×2, axis-angle rotation	`Linear(512→1)`, one score per hypothesis
input	`c_in = 6` (RGB+XYZ), 160×160, `crop_ratio` 1.2	`c_in = 6`, 160×160, `crop_ratio` 1.1

Both use `normalize_xyz` and batch norm. The refiner's transformer heads are `d_model = 512` with `in_proj 1536×512` — one 512-wide projection each for Q, K and V.

Runtime totals: 36.04 M parameters, of which 3.01 M (8.4%) are trained in this project and 33.03 M are pretrained and frozen.

3. Rig and sensing

3.1 Rig geometry, derived rather than chosen

Camera placement is computed from the measured colour intrinsics (`fx ≈ 605 px` at 848×480 → 70.0° × 43.3° FOV) against the depth-noise law below, not selected by eye: standoff **0.70 m**, elevation **35°**, azimuth **±40°** (80° separation), placing the cameras 402 mm above the tabletop, 737 mm apart, at ~2.2 mm depth noise with 981 mm of horizontal coverage against a 600 mm workspace. The workspace is a 1180 × 440 mm surface, which the two tripods clear by 219 mm.

One consequence is load-bearing for calibration validity. The cameras sit on floor tripods and the table is a shared desk, so the tabletop moves between sessions and **silently** invalidates every `T_world_cam` — each fused pose stays self-consistent while being wrong by a fixed offset. Only `T_world_cam` is affected; intrinsics are rigid to the camera body and `T_camA_camB` to the tripods. `calib_agent` therefore has a **session re-registration** mode that re-derives world registration from one board view per camera in ~30 s instead of a 40-view sweep, guarded by a check that makes it safe: `T_camA_camB = T_world_camA⁻¹ · T_world_camB` is invariant to the world frame moving, so recomputing it separates "the table moved" (recoverable) from "a camera moved" (not), and the tool refuses to write in the second case. On synthetic input both scenarios show an identical 13.04 mm of world-frame motion and are distinguished only by this test.

3.2 Sensor error budget

Depth noise against a flat wall, 30 frames per point, matched 28 × 16 cm wall patch at every distance, firmware 5.17.3.10:

distance	cam_front	cam_over
0.4 m	0.98 mm (0.24%)	0.75 mm (0.19%)
0.7 m	2.69 mm (0.38%)	2.44 mm (0.35%)
1.0 m	4.77 mm (0.48%)	4.27 mm (0.42%)

These figures are only comparable across distances because the *physical* wall patch was held constant rather than the pixel ROI; the naive method over-reports by up to 9.7× (Appendix A.1). Both units follow the expected stereo error law — `RMS ≈ 4.9·z²` mm (cam_front) and `4.4·z²` mm (cam_over), power-law exponents 1.73 and 1.92 — and agree with each other to 9–10% at 0.7 and 1.0 m.

At the distances the cameras are mounted (0.60 m and 0.75–0.90 m), depth noise is 2–4 mm. This is the floor for fused-pose accuracy: a fused error at or below it is sensor noise rather than an algorithm deficiency, and the occlusion ablation must be read against it. It is also the target fidelity for the depth channel rendered by Replicator, on which the RGB-D detector depends.

Calibration is captured at 1920 × 1080 while the pipeline streams 848 × 480, because DICT_5X5 markers stop decoding below ~3 px/module and the binding distance is the board's oblique far corner at 884 mm, not the 0.70 m standoff (Appendix A.2).

3.3 Object models and their physical instantiation

Proposal §6.2 summarises this; the full record is kept here.

Acquired 2026-09-04. Three MVP classes: `mustard_bottle` (YCB 006, from the FoundationPose demo data — its 0.196463 m diameter matches the 0.19646 FoundationPose itself reports, confirming it is the mesh the estimator was validated against), `bowl` (YCB 024, `google_16k`), and `red_cube` (an authored 50 mm cube, exact by construction). All are watertight, as FoundationPose requires.

Object-frame convention — the origin is the mesh AABB centre. A scanned mesh's origin is wherever the scanner left it, and FoundationPose reports the pose of *that* origin, so `ObjectState::pose_w.translation()` inherits it. Two downstream assumptions need the centre instead: the scene-graph bounding box is built as `translation() ± extent`, and the `symmetric` flag asserts that rotations about the symmetry axis are equivalent. The raw YCB bowl fails both — its origin lies 43.7 mm off centre (27% of its own diameter) and off the symmetry axis (rim radius sd 30.5 mm about the mesh-

origin axis versus 2.5 mm about the AABB-centre axis). Re-centring on ingest is a rigid re-parameterisation that leaves diameters, extents, ADD/ADD-S and pose accuracy unchanged. `training/prepare_meshes.py` enforces it and two automated tests re-derive it from the mesh files, so `config/meshes.yaml` cannot drift from the geometry it describes.

All three classes are 3D printed. A mesh is only useful if the physical object it describes is the one in front of the cameras: FoundationPose registers the mesh against observed depth and ADD/ADD-S is computed over mesh vertices, so any object/mesh shape mismatch is added to the pose error and cannot afterwards be separated from it. Printing makes the match exact by construction. The cube is authored, so it matches trivially; the YCB bowl mesh is watertight and fully manifold (0 boundary, 0 non-manifold edges) and slices without repair, and printing additionally avoids the original's glossy enamelware, which returns little usable IR to an active-stereo sensor. Purchase was preferred for `mustard_bottle`, whose printed label is the visual signature the RGB half of the detector learns — but the YCB original is French's Classic Yellow **20 oz** (identified from the texture atlas, whose front label reads "40% MORE THAN OUR 14 OZ", and corroborated by the YCB paper's 603 g), and that product is not retail-available in Singapore. Substituting the common 14 oz would reintroduce exactly the mismatch printing exists to prevent, and would do so invisibly: the detector would still fire, FoundationPose would still return a pose, and only ADD-S would silently carry the difference.

The synthetic renders therefore use filament, not the scanned textures. A scanned texture describes an object that will never be photographed, and the bowl's has the scan's own specular highlights baked in, which a renderer would light a second time. `replicator_dataset.py` renders all three in their print colours with `FILAMENT_SPREAD = 0.12` of linear randomisation, so the print colours are fixed by the shipped `synth50k_v2` set rather than chosen at print time; the per-class sRGB targets and acceptable bands are in that script. Matte is a requirement in both directions — gloss defeats the IR projector on the real object (§3.2) and the renderer draws these classes at roughness 0.55–0.90, so a glossy print would depict something the training set does not contain.

Print constraints that affect measurement. Slice at 100% with no bed-fitting rescale, and verify dimensional accuracy with callipers before relying on any object: a printer 1% out of true reintroduces the mismatch printing avoids. The bowl's scanned wall is 0.99 mm — thinner than two perimeters at a 0.4 mm nozzle — and must **not** be thickened for durability, since any deviation from the mesh enters ADD-S and cannot be separated from estimator error. Material per object set, computed from the meshes: 227 g.

Limitation, stated rather than glossed: all three classes are controlled-appearance. Both sides of the sim-to-real comparison are under the project's control, so the measured gap isolates geometry, sensor and lighting effects rather than testing appearance-domain randomisation. That is a cleaner experiment in one sense and a **weaker test of proposal goal G4** in another, and the proposal's §7 reports it as such rather than presenting an artificially small gap as a result. Should the bottle become obtainable, restoring it as a purchased real-texture class recovers the stronger test at no cost to anything else.

3.4 Ground-truth marker trays

Proposal §6.3 summarises this; the full record is kept here.

The marker sits on a tray derived from the object mesh, not glued to the object. A marker attached to a finished object gives ground truth only as good as the subsequent measurement of the marker-to-object offset — an error that enters every ADD/ADD-S figure as a constant bias, and one that is invisible because it leaves all results self-consistent. The offset must come from CAD, not callipers. Cutting a marker seat *into* the objects was rejected for a second reason: `synth50k_v2` was rendered from the **unmodified** meshes, so modifying a target object would introduce a sim-to-real gap the trained detector has never seen, and closing it would mean regenerating 88 GB (`/opt/the-eyes-data/synth50k_v2`) for a change that serves evaluation rather than perception.

`training/author_marker_trays.py` therefore generates one tray per class: a plate whose socket is **boolean-subtracted from that object's own mesh**, so the fit is derived from the geometry rather than measured, plus recessed seats for 40 mm `DICT_5X5_250` markers in the same plate. The ground-truth chain is

`T_cam_object = T_cam_marker · T_marker_tray · T_tray_object`

where only `T_cam_marker` is observed; the other two are chosen in CAD and written to `config/marker_trays.yaml`. The objects themselves are untouched, so `assets/meshes/*.obj` remain the meshes FoundationPose registers against and `config/meshes.yaml`'s `diameter_m`, `extent` and `symmetric` flags remain true statements about the physical objects. All three classes consequently have ground truth that is exact by construction.

The generator verifies what it produces rather than asserting it:

	`red_cube`	`mustard_bottle`	`bowl`
plate (mm)	170 × 90 × 10.5	216 × 100 × 14.5	210 × 210 × 8.5
socket depth	8.0 mm	12.0 mm	6.0 mm
measured socket clearance	0.20 mm	0.12–0.23 mm	0.08–0.12 mm
markers visible per camera	1 of 2	1 of 2	3 of 4
marker ids	100–101	102–103	104–107

Socket clearance is the ground-truth error budget, which is why it is measured rather than nominal: the object can sit anywhere within it, so the worst case — **0.23 mm** — bounds the ground-truth translation error. Against the bottle's 19.6 mm ADD-S threshold that is 1.2%, so it cannot move a verdict. Marker ids are allocated globally (the ChArUco board holds 0–31), because per-tray numbering would make id 100 ambiguous the moment two objects share a scene — which is every scene in the occlusion ablation.

Marker visibility is ray-cast against the seated object at the rig's design 35° elevation and ±40° azimuth rather than argued: each camera sees at least one marker on every tray. Because the two cameras generally read *different* markers on the wing trays, agreement between their independently derived object poses is a free check on the print.

Distractor objects used for clutter and false-positive testing need no meshes, as they are never detected classes. The trays are likewise not a detected class, so they print in one of the three **distractor neutrals** the renderer already samples (§3.3) rather than a target colour — a tray in a class colour would put a large slab of that class's colour directly beneath the object.

3.5 Printed parts: checks so far

All six parts — the three objects and their three trays — and the ArUco marker sheet were printed by 2026-09-14. The checks below are recorded in `docs/PRINT_PLAN.md` §2d and §4; anything not listed there has not been measured.

Dimensional check — red cube, passed 2026-09-13.

axis	nominal	measured	error
X	50.00 mm	49.88 mm	–0.12 mm (–0.24%)
Y	50.00 mm	49.85 mm	–0.15 mm (–0.30%)
Z	50.00 mm	50.10 mm	+0.10 mm (+0.20%)

Worst axis 0.30%, a third of the 1% stop line. No compensation is applied, and `config/meshes.yaml` is unchanged: the mesh stays the reference, and this table is the recorded error of the part against it.

Socket fit — red cube tray, passed 2026-09-13. The cube seats under its own weight with no lateral play perceptible by hand.

socket opening	drawn	measured	realised clearance per side
X	50.40 mm	50.32 mm	0.220 mm
Y	50.40 mm	50.30 mm	0.225 mm

The measured **0.225 mm** replaces the CAD figure as the ground-truth bound for `red_cube` : 2.6% of its 8.66 mm ADD-S threshold. It does not carry over to the other trays, whose sockets are curved; the bowl's is also drawn tighter, at 0.082–0.118 mm.

Marker scale — ArUco sheet, passed 2026-09-13. Both 100 mm scale bars measure 100.0 mm, and a marker square measures 40.00 mm across and down, so `marker_length_m: 0.0400` stands as configured. This check matters more than socket clearance: a marker printed 0.5% large would put every ground-truth pose about 3.5–4.5 mm too close to the camera at the rig's 0.7–0.9 m.

Bowl tray corrected before printing, 2026-09-12. Cutting the socket from the bowl's 0.99 mm shell had left the bowl's interior behind as a loose island of plate material. The generator now removes any body the cut leaves enclosed by the object, and its `--verify` fails on more than one body. The tray fell from 193 g to 176 g, and `config/marker_trays.yaml` — the transform chain and the measured clearances — is byte-identical.

Not yet checked: the bottle and bowl against CAD, the bottle and bowl socket fits, and whether the marker cards sit flush in their recesses.

4. Measured results

4.1 Multi-view fusion: the eight-arm ablation (*synthetic*)

The fusion ablation, run through the real module chain on *synthetic* input (200 frames, 3 objects, 10 mm/axis Gaussian per-view noise, two objects occluded from `cam_over` on 3 of 4 frames). Views, policy (naive mean vs. gated confidence-weighted fuse) and EKF each vary independently:

arm	views	policy	filter	ADD-S (mm)	ADD-S (/diam)	recall @0.1d	missed
cam_front	1	—	raw	9.97	0.1151	0.3517	0
cam_over	1	—	raw	10.37	0.1197	0.1517	300
naive (baseline)	2	naive	raw	9.00	0.1039	0.4850	0
gated	2	gated	raw	9.01	0.1040	0.4833	0
cam_front+ekf	1	—	ekf	6.62	0.0765	0.8550	0
cam_over+ekf	1	—	ekf	7.85	0.0906	0.3183	300
naive+ekf	2	naive	ekf	6.28	0.0725	0.9017	0
fused (the system)	2	gated	ekf	**6.29**	**0.0726**	**0.9017**	**0**

The table is read as pairs of rows that differ in exactly one thing. The second camera's contribution is `naive − cam_front` (9.00 vs 9.97 mm, and recall 0.485 vs 0.352). The EKF's contribution is any `+ekf` row against its raw twin (about 2.7 mm and 0.4 of recall). The fusion **policy's** contribution — the disagreement gate, confidence weighting and symmetry handling, which is what this module is graded on — is `gated − naive` at the raw tier and `fused − naive+ekf` at the EKF tier. On this input both are **0.01 mm**.

That zero is correct, and it is why the design has eight arms rather than four. Under symmetric Gaussian noise with no outliers the gate has nothing to reject and equal per-view scores make the weighted mean the plain mean, so the policy *cannot* win here; a harness showing it win would be measuring something else — most likely the EKF's temporal averaging credited to the spatial policy. The filter is therefore an independent switch in `FusionConfig`, and `Ablation.TheFilterAndThePolicyAreNotConfounded` pins the policy delta at zero and the filter delta large on this input, so that confound cannot return unnoticed. `naive` shares the world-frame lift, the association and the quaternion averaging with `gated` and differs only in gate, weighting and symmetry handling — anything else would manufacture a difference rather than measure one.

Where the policy earns its place is a **confidently wrong view** — a bias, which temporal averaging cannot remove. `GatedFusionBeatsTheNaiveBaselineWhenOneViewIsAnOutlier` injects an 8 cm error into one view and asserts the gated arm wins at *both* filter tiers. Whether FoundationPose produces such views at this rig geometry, and how often, is what the real-capture ablation measures (proposal §7.1).

Two qualifications: the figures are *synthetic* and every tool printing them says so; and `ApplyDisagreementGate`'s three-or-more-view majority branch is unit-tested but unreachable on a two-camera rig, so the gate's real behaviour is entirely the two-view tie-break path.

4.2 First real-object run of the whole chain

On 150 live pairs of a printed cube — with fixed boxes standing in for a detector not yet trained on real data — the two views associated and fused into a single object on **every** pair, zero gate rejections, fused pose stable to 0.1 mm sd, per-view world-pose disagreement 11 mm, `(on red_cube_01 table_01)` holding for all 150 pairs at 22.3 pairs/s.

The one checkable figure is height: a 50 mm cube centres at 25 mm and the fused z reads 29.7 mm, **5 mm high**, consistent with 2–4 mm depth noise plus calibration. *This is one observation, not a measurement, and it is not an ADD-S number.* It is the first evidence on real data that the multi-view claim holds end to end.

Two things the run exposed that the synthetic harness could not:

- **FoundationPose scores are not constant on real frames** (0.75–0.92 across 235 estimates), and differ per view on the same object (0.656 cam_front vs. 0.712 cam_over). Confidence weighting has something to weight after all.
- **A wrong fixed box fails silently.** An earlier attempt used coordinates from a stale capture; the boxes held a mouse and bare table and FoundationPose returned stable, confident poses for 150 pairs. The 181 mm inter-view disagreement was the only tell. `docs/results/fixed_box_stale_failure.png` is all that was kept of that run.

4.3 Detector training, and why mAP@0.5 is the wrong number here

The RGB-D detector has now been trained twice, and the comparison is more informative than either result alone. On the first synthetic dataset it reached **mAP@0.5 = 0.994 after a single epoch** and 0.995 by the tenth, against this proposal's provisional target of 0.90. That is not a result to report; it is a warning that the benchmark cannot discriminate. Inspection confirmed why: three classes in three unmistakable filament colours on a plain table, with 98% of frames showing all three targets essentially unoccluded.

The scenes were rebuilt to be discriminative: distractors drawing from the *same* colour palette as the targets, so colour ceases to be sufficient and shape and depth must carry the decision; eight distractors rather than three; and a tighter workspace so objects crowd. Occlusion is measured rather than assumed, from the ratio of Replicator's tight (visible) to loose (full) boxes — mean occlusion rose from ~0% to 31–35%, objects more than 25% occluded from ~0% to 54–56%, and frames showing all three targets fell from 98% to 62%.

Retrained on the harder data, 40 epochs, 40,012 train / 9,988 val:

class	instances	P	R	mAP@0.5	mAP@0.5:0.95
mustard_bottle	9,988	0.997	0.998	0.995	0.989
bowl	9,322	0.989	0.986	0.994	0.966
red_cube	**6,622**	0.988	**0.963**	0.992	**0.915**
all	25,932	0.991	0.983	**0.994**	**0.956**

Harder scenes did not prevent saturation at mAP@0.5. It converged to 0.994 against the easy dataset's 0.995 — a difference of one thousandth. What changed is everything underneath: mAP@0.5:0.95 fell from 0.985 to 0.956, and convergence slowed from one epoch to roughly eleven.

The consequence for the proposal's evaluation plan (§7) is concrete. **The modality and dataset-size ablations must be reported on mAP@0.5:0.95 and per class, not on mAP@0.5**, which has 0.006 of headroom and can register nothing. mAP@0.5:0.95 retains 0.044, and `red_cube` alone retains 0.085 — that class is the discriminating one, being the smallest object and the most often occluded, visible in only 66% of validation images against the bottle's 100%. A depth channel that helps should help there first, which makes it the natural place to look for the RGB-D gain rather than in a pooled average that is already close to its ceiling.

A third, subtler defect followed from the object-frame convention (proposal §6.2): with the origin at the AABB centre each class rests at its own half-height (96, 25, 28 mm), so one shared placement range buried some classes in the table and floated others — while the 2D boxes stayed correct, so the label files looked perfect and `(on obj table)` was false in the very data the predicates are to be evaluated against.

4.4 The modality ablation — a negative result

This is the ablation that tests the project's own multimodal claim (§4, category ②), so it is reported whichever way it comes out. RGB-only versus RGB-D on the same 9,988 val frames, byte-identical PNGs and labels, identical 40,012/9,988 split, same 40-epoch schedule and seed, `yolov8n.pt` COCO init both sides — differing **only** in whether the loader reads the 3-channel PNG or the 4-channel `.npy`, so any delta is attributable to the depth channel and nothing else:

	RGB-only	RGB-D	delta
mAP@0.5	0.99325	0.99371	+0.00046
mAP@0.5:0.95	0.95662	0.95738	**+0.00076**
recall	0.98272	0.98260	−0.00012
`mustard_bottle` @.5:.95	0.98872	0.98861	−0.00011
`red_cube` @.5:.95	0.91565	0.91590	**+0.00026**
`bowl` @.5:.95	0.96551	0.96763	+0.00212

The depth channel does not earn its place on this benchmark. `red_cube` was the class predicted to move — smallest, most often occluded, and the only one with real headroom at 0.085 below ceiling — and it moved by 0.0003. Two of three classes are marginally *worse* on recall with depth.

The reading is narrow and deliberate: at 100% synthetic depth validity, three geometrically distinct classes in three distinct colours, and a COCO-pretrained backbone, depth adds nothing *this benchmark* needs. That is not a statement about RGB-D detection in general, and it is not "the benchmark is saturated" — the dataset-size ablation below rules that out, since 8x more training data moves the same val split by +0.0322. There is headroom; depth does not reach it while more RGB examples do.

Two consequences follow rather than being argued away. The early-fusion stem stays in the delivered system because the real-capture condition is the one it was chosen for — real depth validity is 97.8%, not 100%, and the appearance gap is real — and that test has not run. And if it does not move there either, the honest finding is that the RGB-D stem was the right design for the stated risk and the risk did not materialise on this object set. `AGENTS.md` set that policy in advance: "if they do not, that is a reportable negative result, not a failure to hide."

Raw metrics: `docs/results/modality_rgb_vs_rgbd_50k_v2.json`.

4.5 The dataset-size ablation, and what it says about the modality result

AGENTS.md makes this an explicit deliverable. All three arms share one generator config, one scene-sampling distribution, one 9,988-frame val split, one seed and one 40-epoch schedule; only `--fraction` differs. That matters: three separate generations would have differed in scene sampling as well as in size, and the comparison could not then attribute a difference to size alone. Arms are named for the dataset they subset — the actual **train** counts are 5,001 / 20,006 / 40,012, because `synth50k_v2` is 50k *total* (40,012 train + 9,988 val).

train frames	mAP@0.5	mAP@0.5:0.95	recall	`red_cube` @.5:.95
5,001	0.98805	0.92520	0.97093	0.86445
20,006	0.99333	0.94695	0.98274	0.89714
40,012	0.99371	**0.95738**	0.98260	0.91590

The prediction made above — that these ablations must be read on mAP@0.5:0.95 and per class rather than on mAP@0.5 — is borne out. mAP@0.5 moves 0.0057 across an 8× change in training data and would have supported no conclusion at all.

Recall saturates at 20k; localisation accuracy does not. Recall is flat past 20,006 frames (0.98274 → 0.98260) while mAP@0.5:0.95 still gains +0.0104 over the same interval. The additional data is not finding more objects — it is placing the boxes it already finds more precisely. `red_cube`, the smallest class and the one with the most headroom, takes most of the

benefit: +0.0514 across the 8× against +0.0322 pooled. The curve has not flattened, so the full generation is justified empirically rather than by assertion, and more data would still buy accuracy — worth knowing before the sim-to-real work, where the temptation under schedule pressure is to generate less.

This qualifies the modality result above. That section attributes depth's non-effect to RGB having already saturated the task. On the same val split, 8× more training data buys +0.0322 pooled and +0.0514 on `red_cube`, so the task is demonstrably not saturated. The defensible statement is narrower than the one first offered: the remaining headroom is in localisation precision, and an early-fusion depth channel does not reach it on this benchmark while more RGB examples do. Both ablations are reported as measured, including where the second constrains the first.

Raw metrics and plot: `docs/results/dataset_size_ablation_rgbd_50k_v2.json` and `docs/results/dataset_size_ablation_rgbd_50k_v2.png`.

4.6 Runtime performance

Stage	Measured	Note
Detector, GPU	**p50 3.60 / 3.43 ms per view**	12.60 / 12.37 ms on CPU; same detections either provider (scores within 0.0024, boxes within 1 px)
Pose, per object	p50 11.1 ms	Real FoundationPose server, 30/30 frames
Pipeline total	7.13 ms	Was 25.14 ms before the GPU provider loaded
Live, end to end	**p50 51.7 ms per pair** (~19 pairs/s)	p95 78 ms, 120 pairs. 8.7 pairs/s over the whole run because first registration took 2.1 s — visible as 56 dropped frames, not hidden
Recording	30 pairs/s, zero drops	PNG level 1, two writer threads; replays bit-exact

`[VERIFY]` End-to-end FPS with three tracked objects per view (risk R4's 9.7 FPS estimate). Now that the detector costs 7 ms rather than 25 ms per pair, FoundationPose is essentially the whole budget.

5. Findings and discussion

F1 — The second camera buys recall; the temporal filter buys accuracy. On synthetic input the second camera moves recall 0.352 → 0.485 while the EKF moves ADD-S by ~2.7 mm. These are separable contributions and the eight-arm design exists to keep them separable.

F2 — The fusion *policy* contributes nothing under symmetric noise, and that is the correct result. `gated − naive` is 0.01 mm. The gate has nothing to reject when no view is an outlier. The policy earns its place only against a confidently wrong view, which is a bias rather than noise; whether the real rig produces such views is the open question (proposal §7.1).

F3 — The depth channel does not earn its place on the synthetic benchmark (§4.4). Reported as a negative result. The real-capture condition, where depth validity is 97.8% rather than 100%, is the one it was chosen for and has not been tested.

F4 — More RGB data still buys accuracy where depth does not (§4.5). This qualifies F3: the task is not saturated, so "depth adds nothing because there is nothing left to add" is not available as an explanation.

F5 — Two failure modes are silent and both were caught by cross-view disagreement, not by any per-view confidence: the stale fixed box (§4.2) and a bumped tripod versus a moved table (§3.1). This is an argument for the multi-view design that the ablation metrics do not capture.

F6 — The `in` predicate asked a question geometry cannot answer. Fixed 2026-09-06. `(in red_cube_01 table_01)` was emitted 26 times in a live run. The rule tested centroid containment and volume ratio, and `ObjectState` carries a world-axis-aligned box and nothing else — which cannot distinguish a container from a slab. A cube pose low by more than half the 20 mm table thickness puts its centroid between the table's faces; the volume ratio stays tiny and every geometric test passes on a false claim.

`in` now requires the container's class to be declared in `predicates.yaml: in.container_classes`, default-deny, checked before any geometry. The shipped list is `[bowl]`. Default-deny because the errors are not symmetric: containment is exclusive in extraction, so a false `in` suppresses every true `on` fact for the contained object, while a false negative forgoes only the `in` fact itself. `unknown_object` never qualifies, which follows from the default rather than a special case.

The regression test asserts that the volume and centroid gates would *not* have rejected the failing geometry, so this cannot later be mistaken for a tolerance that needed widening. Predicate accuracy is no longer blocked on this; like every other real-capture metric, it now waits on the scored captures.

6. What is not yet measured

Real-capture detection mAP, ADD-S on real captures, and predicate accuracy — each of which requires the physical objects and a real calibration. The modality and dataset-size ablations have both run and are reported below — the first as a negative result.

No delivery gates anything, and the rig is calibrated. The sync cable (08-27), ChArUco board (08-28, decode-verified on a real capture) and Isaac Sim 5.1.0 are all in place, meshes acquired 09-04. On 2026-09-06 `calib_agent` ran against the mounted rig, and after the cameras moved it was rerun on 2026-09-09 (RMSE 0.47 / 0.41 px); `runtime_agent` drives the full pipeline live at p50 51.7 ms per pair.

The whole chain has run on a real object. On 150 live pairs of a printed cube — with fixed boxes standing in for a detector not yet trained on real data — the two views associated and fused into a single object on **every** pair, zero gate rejections, fused pose stable to 0.1 mm sd. The one checkable figure is height: a 50 mm cube centres at 25 mm and the fused z reads 29.7 mm, **5 mm high**, consistent with 2–4 mm depth noise plus calibration. *This is one observation, not a measurement, and it is not an ADD-S number* — but it is the first evidence on real data that the multi-view claim holds end to end.

All six printed parts exist as of 2026-09-14 (§3.5). The cube, its tray and the marker scale have passed their checks; the bottle and bowl fit checks and the marker-card flush check remain, then capture → score.

Synthetic scene fidelity was the last blocker and closed on 2026-09-04: depth validity went from 4–15% to **100.0%** and labelled targets per frame from ~0 to **3.00** once a 1 m default near-clipping plane and a missing room were fixed (Appendix A.3). Both defects had produced output that looked entirely correct.

Appendix A — Measurement Methodology

Three measurements in §4 would have produced confident, wrong numbers under the obvious method. How each was made trustworthy is recorded here rather than in the body, because it concerns *how* the figures were obtained rather than what they show.

A.1 Depth RMS: match the physical patch, not the pixel ROI

Depth RMS is comparable across distances only if the *physical* wall patch is held constant, not the pixel ROI — a fixed pixel ROI spans 28 × 16 cm at 0.4 m but 70 × 40 cm at 1.0 m, and once it outgrows the flat surface, edges enter the plane fit and inflate the residual by up to 9.7×. The diagnostic is that the fitted plane's offset and the median point depth agree to within 1 mm when the ROI lies on the wall and diverge by 20–27 mm when it does not.

A.2 Calibration target sizing is a resolution constraint

The ChArUco board (324 × 324 mm, 8 × 8 at 38 mm) carries DICT_5X5 markers, 7 modules across, which stop decoding below ~3 px/module; reach is `fx · marker_length / (3 · modules)`, and the binding distance is the **oblique far corner of the board at 884 mm**, not the 0.70 m standoff:

capture resolution	fx (measured)	px/module	reach
848 × 480 (streaming)	603.43	2.8 — fails	0.82 m
1280 × 720	905.15	4.2	1.23 m
1920 × 1080	**1357.72**	**6.1**	**1.81 m**

Calibration is therefore captured at 1920 × 1080 while the pipeline streams 848 × 480. The recovered camera *pose* is resolution-independent, so a small target suffices provided the capture resolution is chosen deliberately. The tool prints the computed reach before detection, because a board below the decode threshold returns zero corners from images that look entirely correct.

A.3 Replicator scene fidelity

Closed 2026-09-04. The scene renders the derived rig geometry with the measured per-unit intrinsics and the real CAD meshes, verified by a `--diagnose` mode reporting what was actually rendered rather than what the scene description implies:

	before	after
depth validity	4–15%	**100.0%**
labelled targets per frame	~0	**3.00**
nearest rendered surface	1007 mm	**470 mm** (table corner is 458 mm)
invalid-depth sentinel	~85%	**1.5%**

Two defects account for it, and both had produced output that looked correct: the **USD default near clipping plane of 1 m** clipped the entire subject (the camera stands at 0.70 m, the nearest table corner at 0.46 m), and **the scene contained no room**, so 69% of every frame hit no geometry and returned the invalid sentinel — which would have taught the detector that "sentinel" means background, a property the real sensor does not have.